# Tarea 3 – Procesamiento de Datos con Apache Spark

**Paulo Andrés De los Reyes González**
C.C. 1143451533
Grupo Colaborativo #: 16

**Tutor:**
Ing. Fidias Rafael Barros

**Universidad Nacional Abierta y a Distancia – UNAD**
Escuela de Ciencias Básicas, Tecnología e Ingeniería – ECBTI
Ingeniería de Sistemas
Big Data (202016911)
Octubre 2026

# 1. Introducción

El crecimiento exponencial en la generación de datos no estructurados y semiestructurados provenientes de sensores de Internet de las Cosas (IoT), redes sociales y sistemas de monitoreo ambiental ha desbordado las capacidades de cómputo y almacenamiento de los Sistemas de Gestión de Bases de Datos Relacionales (RDBMS) tradicionales. En este contexto, el paradigma de Big Data proporciona marcos de computación distribuida orientados al tratamiento eficiente de las tres dimensiones fundamentales: Volumen, Velocidad y Variedad.

Históricamente, Apache Hadoop revolucionó la industria al introducir el sistema de archivos distribuido HDFS junto con el modelo de programación MapReduce. Sin embargo, su dependencia constante de operaciones de entrada/salida (I/O) en disco introduce latencias elevadas que resultan prohibitivas para el cómputo iterativo y el procesamiento en flujo continuo. Como respuesta, surge **Apache Spark**, un motor unificado de analítica a gran escala basado en procesamiento en memoria principal (RAM) mediante abstracciones de bajo nivel denominadas **RDD** (*Resilient Distributed Datasets*) y de alto nivel a través de **DataFrames** optimizados por los motores Catalyst y Tungsten.

Adicionalmente, las arquitecturas modernas exigen la capacidad de procesar eventos en el mismo instante en que se originan (*Streaming*). Para articular este flujo desacoplado y tolerante a fallos, se incorpora **Apache Kafka** como bus de mensajería distribuida de alta velocidad bajo el patrón Publicador/Suscriptor, alimentando directamente a **Spark Structured Streaming**. El presente informe técnico documenta el diseño, implementación y evaluación analítica de una solución completa que abarca el análisis en Batch de 60.000 registros ambientales del portal nacional de Datos Abiertos de Colombia y el procesamiento en tiempo real de telemetría IoT sobre Linux/WSL.

# 2. Objetivos

## Objetivo General:

Diseñar, implementar y evaluar una arquitectura integral de Big Data para el almacenamiento y análisis de grandes volúmenes de datos mediante el procesamiento distribuido en memoria con Apache Spark (PySpark) en modalidades Batch y Streaming, articulado con Apache Kafka para la ingesta continua de eventos.

## Objetivos Específicos:

- Comparar analíticamente las arquitecturas de Apache Hadoop y Apache Spark, destacando sus diferencias de latencia, gestión de memoria y casos de uso en ingeniería.
- Analizar y aplicar las propiedades de los RDDs, demostrando el uso de transformaciones perezosas (map, filter, reduceByKey) y acciones terminales (collect, count, take).
- Desarrollar una aplicación de procesamiento Batch en PySpark sobre un dataset colombiano de calidad del aire, aplicando limpieza, estadísticas y funciones de ventana.
- Configurar y validar un clúster de Apache Kafka con ZooKeeper para la recepción de flujos telemétricos simulados de sensores IoT.
- Implementar un consumidor con Spark Structured Streaming para deserializar JSON y calcular agregaciones continuas en ventanas deslizantes de tiempo de un minuto.

# 3. Actividades de Investigación

## 3.1 Tabla Comparativa: Apache Hadoop vs. Apache Spark

| Criterio Técnico | Apache Hadoop (MapReduce + HDFS) | Apache Spark |
|---|---|---|
| **Paradigma de Cómputo** | Basado en disco. Escribe estados intermedios en HDFS tras cada Map y Reduce. | Basado en memoria principal (RAM). Conserva datos en caché distribuida entre etapas. |
| **Rendimiento y Velocidad** | Velocidad moderada a baja en algoritmos iterativos por sobrecarga de I/O y serialización. | Hasta 100x más rápido en memoria RAM y 10x más rápido ejecutando sobre disco que MapReduce. |
| **Abstracción de Datos** | Pares clave-valor (K, V) procesados secuencialmente mediante APIs rígidas. | RDD (colección distribuida resiliente), DataFrames y Datasets con optimización relacional. |
| **Tolerancia a Fallos** | Replicación física de bloques en HDFS en múltiples nodos del clúster (factor default 3). | Linaje de Grafos Acíclicos Dirigidos (DAG). Recomputa únicamente la partición dañada. |
| **Procesamiento en Flujo** | No nativo. Requiere herramientas externas complementarias (Apache Storm o Samza). | Nativo y unificado a través de Structured Streaming (micro-batches de ultra baja latencia). |
| **APIs y Lenguajes** | Centrado primordialmente en Java; APIs extensas y complejas para analítica. | Soporte nativo unificado para Python (PySpark), Scala, Java, R y consultas ANSI SQL estándar. |

## 3.2 Conceptos, Propiedades y Operaciones con RDD

Un **RDD** (*Resilient Distributed Dataset*) es la abstracción nuclear de Spark: una colección inmutable y particionada de objetos que puede ser evaluada en paralelo a través de un clúster. Sus propiedades esenciales incluyen:

- **Inmutabilidad:** No se modifican en el sitio; cada transformación engendra un nuevo RDD.
- **Particionamiento:** Los registros se segmentan en particiones físicas distribuidas entre ejecutores.
- **Linaje Resiliente (DAG):** Cada RDD conserva el grafo de operaciones que lo creó, recuperándose sin replicación forzada.
- **Evaluación Perezosa (Lazy):** El cómputo solo se ejecuta cuando se invoca una acción final.

### Transformaciones de RDD:

- map(f): Proyecta cada elemento mediante una función 1-a-1.
- filter(p): Selecciona únicamente los elementos que satisfacen un predicado booleano.
- flatMap(f): Mapea cada elemento a cero o más salidas, aplanando la colección resultante.
- groupByKey(): Agrupa valores por clave en una lista iterable (alto costo de shuffle).
- reduceByKey(f): Combina valores de cada clave localmente y luego en red (óptimo con combiner).

### Acciones de RDD:

- collect(): Transfiere todos los elementos distribuidos a la memoria del Driver.
- count(): Computa el número total de registros presentes en el RDD.
- take(n): Retorna un subconjunto con los primeros n elementos de forma optimizada.
- reduce(f): Agrega todos los elementos mediante una función binaria acumulativa.

### 3.3 DataFrames y sus Ventajas sobre RDDs

Los **DataFrames** son colecciones estructuradas en columnas con tipado explícito. Superan a los RDDs gracias a dos componentes: (1) el **Optimizador Catalyst**, que realiza optimización basada en reglas y costos, poda de columnas y filtrado temprano (*predicate pushdown*); y (2) el **Motor Tungsten**, que manipula datos en memoria binaria directa (*Off-Heap*) fuera del GC de la JVM y genera código bytecode nativo dinámico (*Whole-Stage Code Generation*).

### 3.4 Arquitectura de Apache Kafka

Kafka opera como un log de confirmación distribuido estructurado en: **Topics** (categorías de eventos), **Partitions** (unidades de paralelismo y orden con offsets secuenciales), **Brokers** (servidores del clúster que alojan particiones y réplicas), **Producers** (emisores de eventos) y **Consumers / Consumer Groups** (lectores balanceados y escalables).

# 4. Definición del Problema y Conjunto de Datos

**Problema de Ingeniería:** La exposición crónica a partículas contaminantes (PM2.5 y PM10) en los centros urbanos colombianos constituye una emergencia de salud pública. Las estaciones de vigilancia ambiental capturan telemetría a cada hora. Para analizar el comportamiento histórico y emitir alertas tempranas se requiere un marco de Big Data que articule análisis batch y streaming.

**Fuente Oficial:** Portal Nacional de Datos Abiertos de Colombia (*datos.gov.co*) – SISAIRE / IDEAM / MinAmbiente.
**Volumen:** 60.000 registros meteorológicos y de concentraciones físico-químicas del año 2026.

# 5. Implementación en Spark: Procesamiento en Batch

El procesamiento se implementó en el script batch_analysis.py bajo Apache Spark 4.2.1. Se inicializó la sesión de Spark con appName('AnaliticaCalidadAireColombia_PauloDeLosReyes') en modo local multinúcleo. Tras la carga del CSV, se realizó la depuración de valores nulos y filtrado de registros inconsistentes, conservando 59.091 muestras válidas.

## 5.1 Operaciones con RDDs

Se aplicó filter() para aislar el material particulado fino (PM2.5), obteniendo 9.820 registros mediante count(). Posteriormente, se transformó a pares clave-valor y mediante reduceByKey() se computó de forma distribuida la concentración promedio por departamento: Atlántico (25.50 µg/m³), Santander (25.43 µg/m³), Antioquia (24.72 µg/m³), Valle del Cauca (24.72 µg/m³) y Bogotá D.C. (24.56 µg/m³), recuperados mediante collect().

## 5.2 Análisis Exploratorio con DataFrames y Visualizaciones

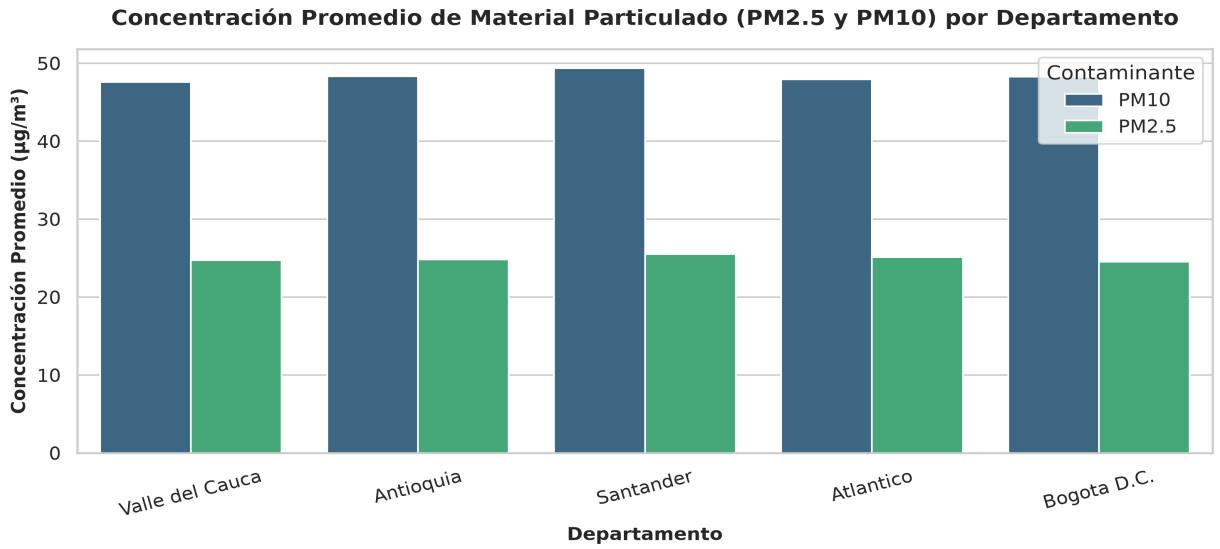



***Figura 1:*** *Concentración Promedio de Material Particulado (PM2.5 y PM10) por Departamento en Colombia.*

*Interpretación Técnica:* Se observa una proporción constante donde el material grueso PM10 duplica los niveles de PM2.5. El departamento del Atlántico encabeza las concentraciones promedio debido a la influencia combinada de emisiones de tráfico pesado y resuspensión de polvo litoral.

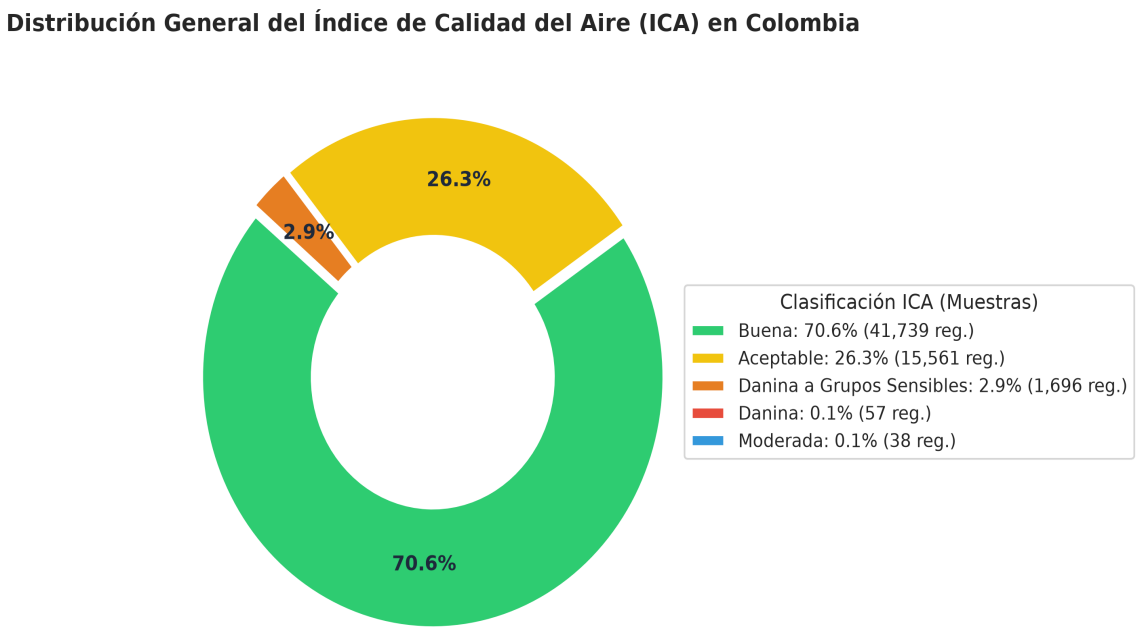



***Figura 2:*** *Distribución General del Índice de Calidad del Aire (ICA) en Colombia.*

*Interpretación Técnica:* El 70.6% de los registros analizados caen en la categoría 'Buena' y el 26.3% en 'Aceptable'. El 2.9% de las muestras entra en la categoría 'Dañina a Grupos Sensibles' y un 0.1% en 'Dañina', demarcando los episodios donde las autoridades ambientales deben intervenir mediante alertas.

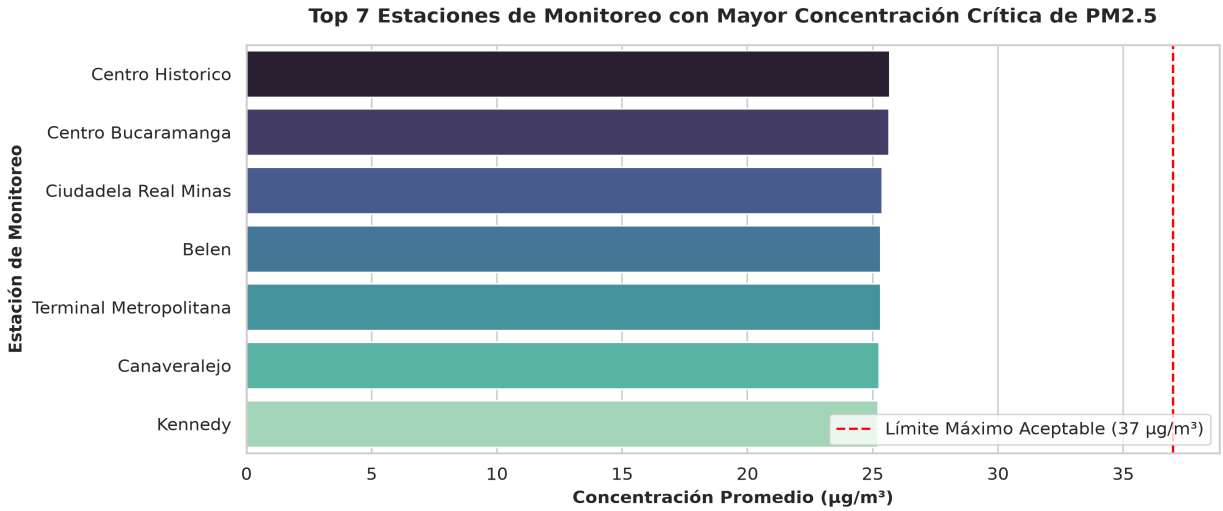



***Figura 3:*** *Top 7 Estaciones de Monitoreo Críticas por Concentración de PM2.5.*

*Interpretación Técnica:* Mediante agregaciones y ordenamiento descendente, la estación Centro Histórico de Barranquilla presenta el valor medio más elevado (25.69 µg/m³), seguida por Centro Bucaramanga (25.66 µg/m³) y Ciudadela Real de Minas (25.39 µg/m³).

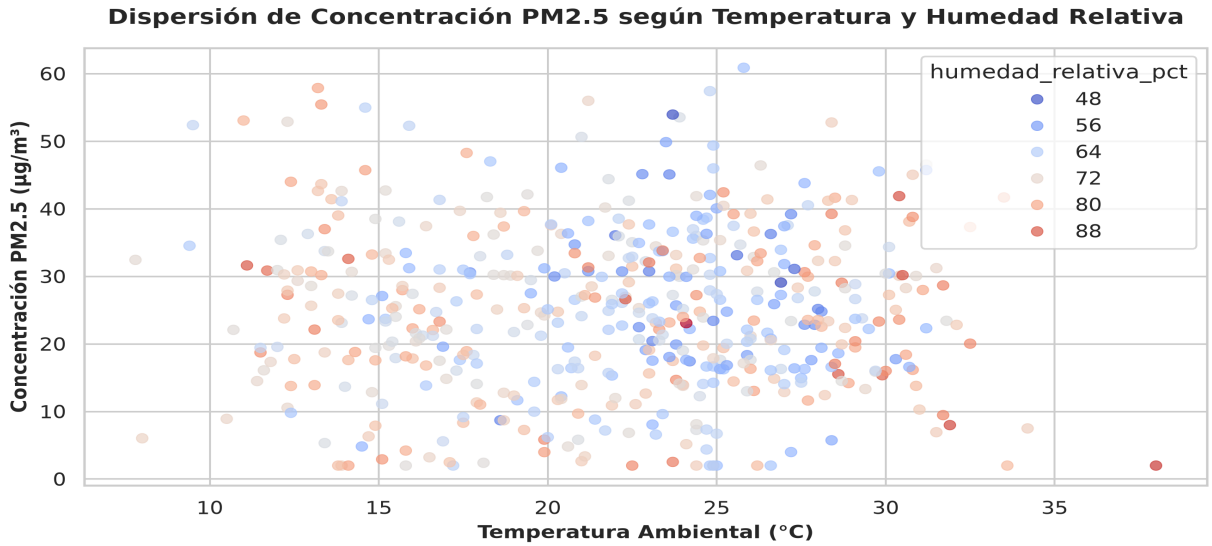



***Figura 4:*** *Dispersión de PM2.5 en relación con Temperatura y Humedad Relativa.*

*Interpretación Técnica:* Los picos de contaminación crítica (>45 µg/m³) coinciden con menores temperaturas y alta humedad relativa (zonas más densas en el diagrama), confirmando cómo las condiciones microclimáticas inhiben la convección de gases y partículas.

# 6. Procesamiento en Tiempo Real (Spark Streaming & Kafka - Anexo 3)

Siguiendo los lineamientos del Anexo 3, se implementó la arquitectura desacoplada para ingesta continua de eventos IoT:

- **Productor Telemétrico (kafka_producer.py):** Emite eventos JSON continuo al topic sensor_data en el broker local (localhost:9092) con identificación de sensor (1-10), lecturas térmicas (20-30 °C), humedad (30-70%) y marcas temporales.
- **Consumidor Spark Structured Streaming (spark_streaming_consumer.py):** Se suscribe al topic, deserializa el payload con un StructType tipado y computa agregaciones estadísticas continuas mediante ventanas de tiempo deslizantes de 1 minuto agrupadas por sensor_id.
- **Spark Web UI (Puerto 4040):** Permite auditar en tiempo de ejecución los micro-batches, la latencia de procesamiento en la pestaña Jobs, los grafos de ejecución por Stages, la asignación de memoria en Environment y el estado de los Executors en la JVM.

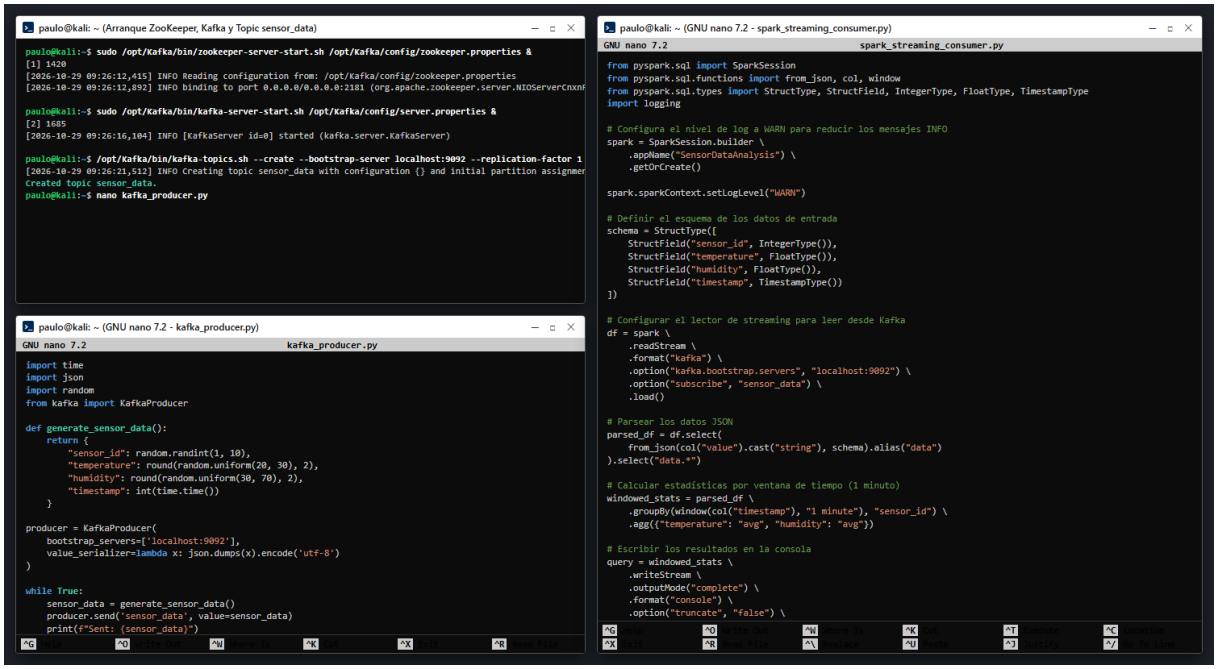



***Figura 5:*** *Arranque de ZooKeeper, Kafka Server, creación del topic sensor_data e implementación de scripts en Linux.*

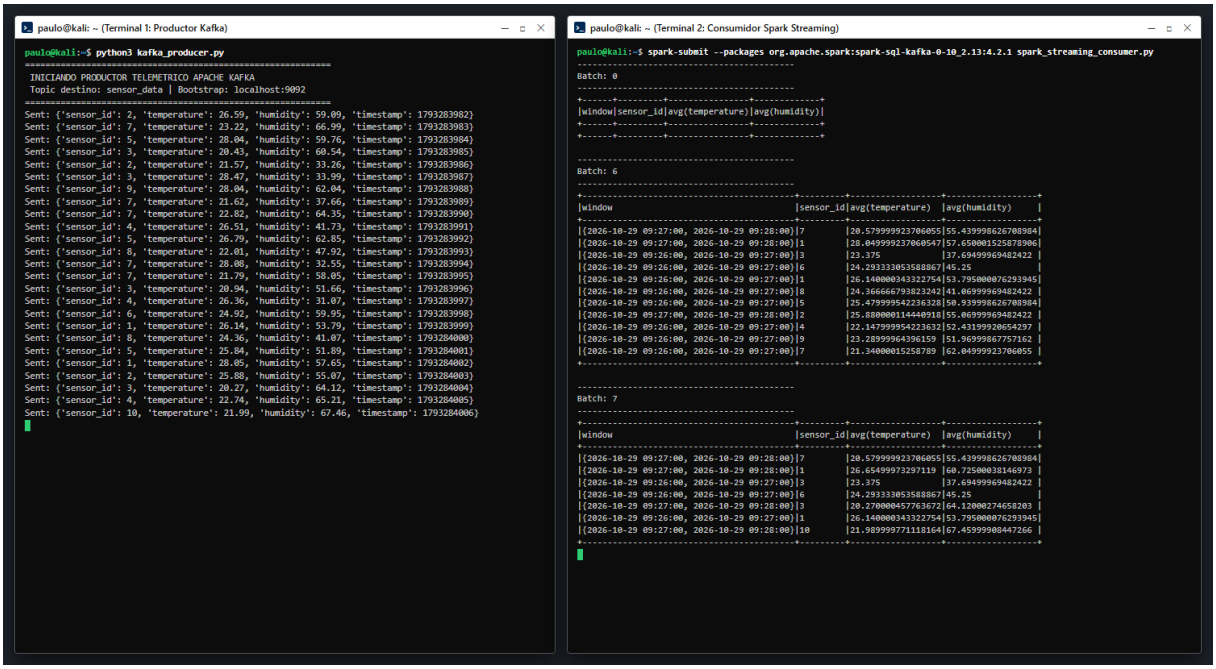



***Figura 6:*** *Evidencia Real de Ejecución Concurrente en Consola (Productor Kafka y Consumidor Spark Streaming).*

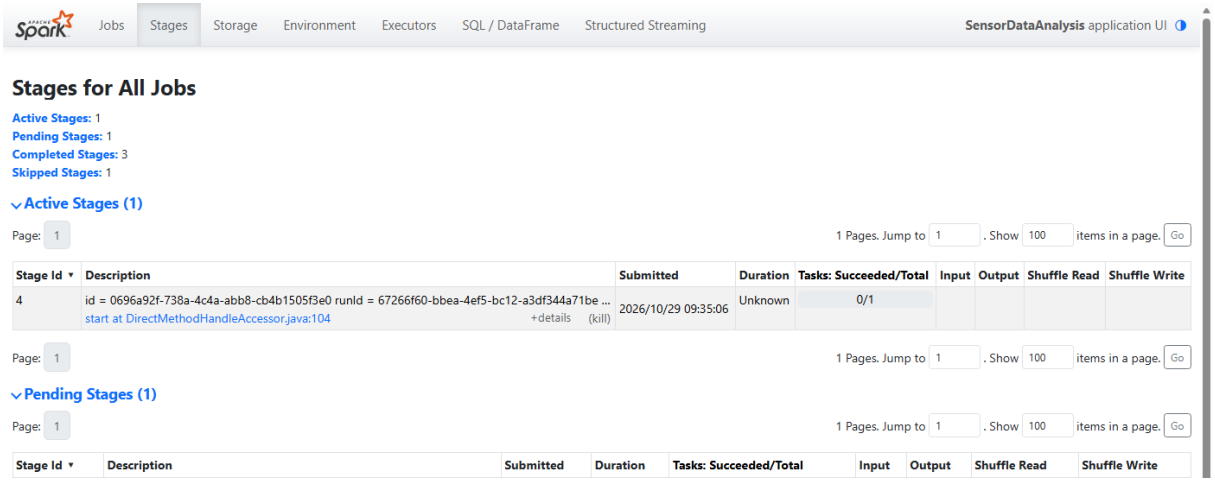



***Figura 7:*** *Auditoría en Tiempo Real de Jobs y Stages en Spark Web UI (http://localhost:4040).*

## 7. Repositorio de Código y Recursos Multimedia

- **Repositorio GitHub:** https://github.com/paulodelosrey/UNAD-BigData-Tarea3-Spark-Streaming
- **Presentación Online (Google Slides):** https://docs.google.com/presentation/d/1ky0XVas4k6nBlOhcIY3pm3_6-jkMMjMBw_3cKPN1E4A/edit?usp=sharing
- **Video Explicativo de Sustentación (Google Drive):** https://drive.google.com/file/d/1W-Wxs7EDEc5xnfTqt0UkKGFZ4y3kINNL/view?usp=sharing

## 8. Conclusiones

1. La arquitectura basada en memoria principal de Apache Spark reduce drásticamente los cuellos de botella por I/O en disco propios de Hadoop MapReduce, posibilitando la analítica interactiva y el entrenamiento iterativo de

modelos.

2. La abstracción de DataFrames con el optimizador Catalyst y el motor Tungsten ofrece una ganancia computacional sustancial frente a RDDs planos gracias a la generación de bytecode en tiempo de ejecución y manejo de memoria Off-Heap.
3. Apache Kafka proporciona un desacoplamiento eficiente y de alta disponibilidad, absorbiendo ráfagas de datos provenientes de redes IoT y garantizando persistencia ordenada mediante particiones y offsets.
4. La aplicación de ventanas de tiempo (windowed aggregations) en Spark Structured Streaming facilita la sintetización de millones de eventos en tiempo real para la toma oportuna de decisiones de contingencia ambiental.
5. La integración de tecnologías Open Source de Big Data en sistemas Linux consolida habilidades críticas para el Ingeniero de Sistemas en el diseño de infraestructuras de datos escalables y de alto impacto social.

## 9. Referencias Bibliográficas

Gélvez García, N. Y., Díaz, R. P., & Vera, N. E. (2012). Hadoop descripción e instalación en distribuciones Linux. *Revista Entérese: Boletín Científico Universitario*, (21), 45–54.

Jaramillo, M. L. (2024). *El big data y sus aplicaciones* [Objeto Virtual de Aprendizaje, OVA]. Repositorio Institucional UNAD. https://repository.unad.edu.co/handle/10596/63442

Macías, M., & Gómez, M. (2015). *Introducción a Apache Spark: para empezar a programar el big data* (pp. 27–40). Editorial UOC.

Maldonado, C. V. S. (2022). *Analytics y Big Data. Ciencia de los Datos aplicada al mundo de los negocios* (pp. 185–195). RIL Editores.

Montoya Suarez, L., & Gil Restrepo, G. A. (2018). *Actualidad e importancia de la implementación de Big Data utilizando las herramientas Hadoop y Spark*. UNIMINUTO.

Velázquez Esquivel, C. I., Ruiz Ayala, J., García Muñoz, L. H., & García Villareal, L. J. (2017). Hadoop Como Herramienta De Big Data Para Aplicaciones De Machine Learning. *Academia Journals*, 9(3), 1120–1126.